SECTION

2. Battle of definitions

Math writing vs mathlib: a culture shock.

Formalizing finitely presented groups

DEFINITION: Finitely presented group

A group G is **finitely presented** if there exist a **finite** set of generators S and a **finite** set of relations R such that

$$G \cong \langle S \mid R \rangle := F(S)/\langle\langle R \rangle\rangle$$

where $F(S)$ is the free group on S and $\langle\langle R \rangle\rangle$ is the normal closure of R.

Main issue: there are many valid ways to formalize this!

Going from set theory to type theory wasn't trivial either.

Using a surjection (due to Riccardo Brasca)

DEFINITION: Finitely presented group

$$\exists S, R \text{ finite such that } G \cong F(S)/\langle\langle R \rangle\rangle$$

LEAN

```
class IsFinitelyPresented (G : Type*) [Group G] : Prop where
  out : ∃ (n : ℕ) (f : FreeGroup (Fin n) →* G),
    Function.Surjective f ∧ IsNormalClosureFG f.ker
```

- S is `Fin n`, $\langle\langle R \rangle\rangle$ is `f.ker`
- "`Fin n` *is a natural number i with the constraint that i < n. It is the canonical type with n elements.*" docs: Init/Prelude/Fin

This is the definition we started with at ItaLean.

Using a surjection - Part 2

```
class IsFinitelyPresented (G : Type*) [Group G] : Prop where
  out : ∃ (n : ℕ) (f : FreeGroup (Fin n) →* G),
    Function.Surjective f ∧ IsNormalClosureFG f.ker
```

LEAN

Feedback from project teammates: (also mathematicians and beginners)

- **1**: *"This isn't really what a finitely presented group is."*
- **2** *"Do you even know what a finitely presented group is?!"* (hyperbole)

Our code at the end of the workshop.

- Feels like the start of a nice independent project but not mathlib-ready.
  - There are no proofs, just definitions which is a no-no for new contributors.
- I decide to re-work it over Christmas break with the goal of upstreaming.

Using isomorphisms (mine)

DEFINITION: Finitely presented group

$$\exists S, R \text{ finite such that } G \cong F(S)/\langle\langle R \rangle\rangle$$

LEAN

```
class IsFinitelyPresented (G : Type*) [Group G] : Prop where
  out : ∃ (α : Type) (_: Finite α) (rels : Set (FreeGroup α)) (_ : rels.Finite),
  Nonempty (G ≃* (PresentedGroup rels))
```

- S is `(α : Type) (_: Finite α)`, R is `rels : Set (FreeGroup α)`

Things felt smooth and easy-ish because that's what I was used to.

Using isomorphisms - Part 2

```
class IsFinitelyPresented (G : Type*) [Group G] : Prop where
  out : ∃ (α : Type) (_: Finite α) (rels : Set (FreeGroup α)) (_ : rels.Finite),
  Nonempty (G ≃* (PresentedGroup rels))
```

LEAN

I wrote a lot of code with this!

- Code to go between the different definitions + isomorphism theorem.
- `FP_is_FG` instance, `FreeGroup` instance, trivial instance, mul $\mathbb{Z}$ instance, etc

USE OF AI

I used Aristotle, then GPT-5.2, 5.3 and 5.4 to do this (will come back to this).

Using isomorphisms - Part 3

```
class IsFinitelyPresented (G : Type*) [Group G] : Prop where
  out : ∃ (α : Type) (_: Finite α) (rels : Set (FreeGroup α)) (_ : rels.Finite),
  Nonempty (G ≃* (PresentedGroup rels))
```
LEAN

Review: *"both the quantification over* `Type` *as wells as the* `Nonempty` *strike me as a little strange for the main definition."*- Source.

WHAT I LEARNED

- **Math:** *"a group is FP because ∃ isomorphism to FP group"* is fine.
- **Lean:** have to deal with a non-specific isomorphism and unspecific `Type` (or `Type*`) which can cause universe issues *as a default*.

Using isomorphisms - Part 4: Type digression

Universe hierarchy in CIC to avoid Girard's paradox [2] (the type-theoretic analogue of Russell's paradox: `Type : Type` is inconsistent).

- **Consequence:** everything needs to be grounded at a universe level.

WHAT I LEARNED

- It's enough to say "*for some* `Type`" in math, but not Lean. Demo.

LEAN

```
    Group.FG G ↔ ∃ (α : Type*) (_ : Fintype α) (φ : FreeGroup α →* G),
Function.Surjective φ := by
```

- `←` You're providing a type of arbitrary universe level.
- `→` Lean has to figure out the universe level with nothing to go on!

Using generating set `S : Set G` (due to Thomas Browning)

DEFINITION: Finitely presented group

$$\exists S, R: \quad G \cong F(S)/\langle\langle R \rangle\rangle, \quad |S|, |R| < \infty$$

LEAN

```
class IsFinitelyPresented (G : Type*) [Group G] : Prop where
  out: ∃ S : Set G, S.Finite ∧ Subgroup.closure S = ⊤ ∧
  IsNormalClosureFG (FreeGroup.lift ((↑) : S → G)).ker
```

- S is `S : Set G`, $\langle\langle R \rangle\rangle$ is `f.ker`

Easily extractable data, `S` guaranteed to be in the same universe as `G`.

Use gen set `S : Set G` - Part 2

```
class IsFinitelyPresented (G : Type*) [Group G] : Prop where
  out: ∃ S : Set G, S.Finite ∧ Subgroup.closure S = ⊤ ∧
  IsNormalClosureFG (FreeGroup.lift ((↑) : S → G)).ker
```

LEAN

I wrote a minimal API for this. (`d59b5c3`)

- This definition.
- Keeps the `IsNormalClosureFG.map` property.
  - That is, being FG in the normal closure is invariant under surjection.
- Wrote isomorphism invariance theorem and factored things this time!
  - I was so proud of myself!

`S : Set G` Part 3: commutative diagram for the kernel transport

$$
\begin{tikzcd}
F(S) \arrow[d, "\texttt{FreeGroup.lift ((↑) : S → G)}"'] & F(\varphi(S)) \arrow[l, "\texttt{FreeGroup.freeGroupCongr (iso.toEquiv.image S).symm)}"'] \arrow[d, "\texttt{FreeGroup.lift ((↑) : ↥(↑iso '' S) → H}"] \\
S \subseteq G \arrow[r, "\texttt{iso}"] & \varphi(S) \subseteq H
\end{tikzcd}
$$

`(S : Set G)` `(↑iso '' S : Set H)`

LEAN

```
lemma FreeGroup.lift_mulEquiv_image (iso : G ≃* H) (S : Set G) :
  FreeGroup.lift ((↑) : ↥(↑iso '' S) → H) =
    (↑iso : G →* H).comp ((FreeGroup.lift ((↑) : S → G)).comp
      (FreeGroup.freeGroupCongr (iso.toEquiv.image S).symm)) := by
ext ⟨_, s, hs, rfl⟩; simp [Equiv.image]
```

`S : Set G` Part 4: Feedback

```
lemma FreeGroup.lift_mulEquiv_image (iso : G ≃* H) (S : Set G) :
  FreeGroup.lift ((↑) : ↥(↑iso '' S) → H) =
    (↑iso : G →* H).comp ((FreeGroup.lift ((↑) : S → G)).comp
      (FreeGroup.freeGroupCongr (iso.toEquiv.image S).symm)) := by
ext ⟨_, s, hs, rfl⟩; simp [Equiv.image]
```

LEAN

Review: *"The fact that S is coerced from a term to a type has caused to you have to write the horrible* `FreeGroup.lift_mulEquiv_image` *which does not really look like a library lemma; it is not clean, it looks more like a hack written to dig yourself out of a hole, which should be a red flag."* - Source.

- Gemini 3.1 Pro explains the messy coercions to me.
- Reviewer proposes what we had from the start.

Back at the start, but learned something

```
class IsFinitelyPresented (G : Type*) [Group G] : Prop where
  out : ∃ (n : ℕ) (f : FreeGroup (Fin n) →* G),
    Function.Surjective f ∧ IsNormalClosureFG f.ker
```

LEAN

WHAT I LEARNED

Why was this definition good? Best of both `=*` and `S : Set G` approach.

- No `Nonempty`.
- No `Type` or `Type*` universe shenanigans with generating sets.
- No coercions, which makes API use easier. Potentially avoids diamond.
- Isomorphism-invariance proof still flows. Demo

Big thanks to everyone who guided me

USE OF HUMAN INTELLIGENCE

- **ItaLean2025 organizers:** for giving me the space to start.
- **Riccardo Brasca:** for getting me started.
- **Kevin Buzzard:** for his continued guidance over Zulip and Github.
- **Nehal Patel:** for investigating beginner struggles with me.
- **Thomas Browning:** for reviewing and rewriting my PRs with me.
- **Bhavik Mehta:** for teaching me type polymorphism.
- **Michael Rothgang:** for tips and reviews.
- **Sidharth Hariharan** & **Auguste Poiroux:** for last minute talk prep help.